

Rapport Machine Learning

Lucas TERRASSON - Master Image / IMAC

April 2026

1 Présentation du projet

Introduction :

Ce projet vise à développer un modèle d'intelligence artificielle capable de reconnaître automatiquement le "*type*" de carte à partir d'une image.

Étant joueur régulier du jeu de cartes "Yu-Gi-Oh!", et connaissant les outils me permettant d'extraire des données exploitables, je me suis lancé dans la création d'une base de données d'images de cartes. Mais comment différencier les types de cartes ?

Voici un exemple de carte :



FIGURE 1 – Exemples de cartes

On différenciera trois grandes classes de cartes :

- Les cartes Monstres, principalement de contour marron. Ils ont des étoiles associés (en dessous du titre de la carte) et de nombreux attributs (entre crochets en dessous de l'illustration).
- Les cartes magies de contour vert.
- Les cartes pièges de contour rose.

L'idée pour mon expérience sera de se concentrer sur les attributs des cartes monstres, bien souvent liés à leurs illustrations. En effet, dans notre Figure 1, notre monstre est désigné comme de type : Dragon. Et l'illustration représente bien un dragon. Cependant il arrive que certains monstres ne soit pas représentatifs de leurs types, il y aura donc du tri à faire. Voici un exemple, même monstre avec un type Plante d'un côté et Dragon de l'autre :



FIGURE 2 – Cartes peu différenciables de types différents

Cependant, afin de voir si mon modèle pourrait détecter des détails, j'ai aussi sélectionné des cartes magies particulières. Elles sont dites cartes "Magies Jeu-Rapide", "Magie de Terrain" ou autre. Elles sont différenciables seulement à l'aide du symbole à côté de l'écriture [SPELL CARD] comme vous pouvez voir sur les exemples suivants :



FIGURE 3 – Magie Jeu Rapide / Terrain

Mon intuition reste que même avec un tri des cartes, les illustrations étant très variés, le modèle aura sûrement du mal à trouver des corrélations. Cependant, peut-être pourra-t-il identifier le texte qui détail les types des cartes sur l'image ce qui lui permettrait d'avoir une bonne accuracy.

Objectif :

- Récouter et créer une base de données d'images de jeu de cartes.
- Entraîner un modèle
- Chercher comment optimiser le modèle.
- Conclure quant à l'efficacité d'une intelligence d'artificielle sur ce jeu de données.

2 La base de données d'images

2.1 Volumétrie

Détails :	Valeur :
Nombre total d'images collectées	13281 images
Nombre d'images exploitables après nettoyage	12887 images
Nombre de classes (types)	28 classes différentes sélectionnées
Résolution utilisée (version "small")	268 × 391
Résolution utilisée (version "haute qualité")	813 × 1185
Taille totale du dataset (small)	379,6 Mo (après trie)
Taille totale du dataset (haute qualité)	2,17 Go
Taille moyenne par image (small)	30 Ko
Taille moyenne par image (haute qualité)	160 Ko

2.2 Ressource utilisé

La base d'images n'a pas été totalement constituée à la main image par image. Je savais que je pouvais faire appelle à l'API du site YGOProdeck qui est une référence dans le domaine. Elle répertorie chaque cartes, et leurs bases de données donne un pannel d'informations assez large sur chaque cartes. Chaque carte est référencé par un lien et un id.

Cependant en approfondissant mes recherches, j'ai pu trouver une database regroupant et associant les cartes dans un fichier .csv sur le site Kaggle. Je suis donc partie de cette source.

La recherche de cette banque de donnée m'a pris dans les environs de 1 à 2 heures vu que j'en ai trouvé plusieurs dont des incomplètes. J'ai du passer une vérification un peu fastidieuse pour voir si elle comprenait une base de cartes récente, complète avec des images utilisables (certaines utilisés des sites autre que YGOProdecks).

J'ai ensuite chercher à extraire les données de manières intelligente. En effet, il y a plein de labels différents. En analysant cette BDD, on peut en extraire les labels qui vont nous nous intéresser :

- name (pour identifier les cartes)
- type (Carte Monste, Magie ou Piège)
- race (attribut de notre carte, l'information que notre modèle doit trouver).
- atk, def, level (Des infos complémentaires comme l'attaque, la défense ou le niveau du monstre pour des potentiels tests).
- image_url_small (les liens des images de taille réduite).
- image_url (les liens des images de meilleur qualité).

En effet, j'ai accès à deux qualités d'images différentes fournis par le site. Je vais donc pouvoir tester si la qualité d'image influence sur l'apprentissage.

2.3 Mise en place de la base de donnée

Afin d'extraire ces données, j'ai utilisé du JavaScript pour la simplicité. Avec nodeJS, j'ai pu me dérouiller à faire des fichiers "outils" qui m'aiderait à construire ma BDD.

2.3.1 Télécharger les images en local :

Mon premier fichier (download-card-images.js) me télécharge tous les liens contenus dans mon csv. Il exporte tout dans un dossier "images". En parallèle, je l'ai aussi paramétré afin qu'il puisse me télécharger les images de moins bonne qualité par défaut, ou me télécharger les images de qualité avec une option.

Commandes :

- Image Low Res : `'node download-card-images.js'`
- Image Good Res : `'node download-card-images.js -better'`

De plus ce fichier va permettre de garder la trace des données de chaque images en donnant comme titre à chaque image ses caractéristiques tel que : `name_type_race_atk_def_level`.

Temps : Mise en place du .js : 2 heures (pas mal d'erreurs à corriger et de recherche). Téléchargement de la base : pas plus de 10min pour les deux qualités.

2.3.2 Analyse de la base :

En effet, il faut maintenant voir combien de types différents nous avons et savoir ce qu'on veut garder ou pas. Pour ça, j'ai créé un deuxième fichier JS (count-image.js) qui va me permettre de me rendre compte de tout ça.

En le faisant tourner, j'ai donc ceci :

Listing 1 – Liste des classes

1. normal	: 2237 images
2. warrior	: 1087 images
3. machine	: 993 images
4. continuous	: 985 images
5. fiend	: 850 images
6. spellcaster	: 727 images
7. dragon	: 720 images
8. fairy	: 546 images
9. quick-play	: 495 images
10. beast	: 398 images
11. winged-beast	: 332 images
12. field	: 296 images
13. aqua	: 276 images
14. equip	: 273 images
15. cyberse	: 271 images
16. insect	: 267 images
17. rock	: 256 images
18. plant	: 255 images
19. zombie	: 253 images
20. beast-warrior	: 246 images
21. psychic	: 183 images
22. reptile	: 180 images
23. counter	: 162 images
24. pyro	: 154 images
25. thunder	: 143 images
26. fish	: 142 images
27. dinosaur	: 138 images
28. wyrm	: 94 images
29. sea-serpent	: 90 images
30. ritual	: 78 images
31. illusion	: 24 images
32. yami-yugi	: 6 images
33. bonz	: 5 images

34.	pegasus	: 5 images
35.	divine—beast	: 5 images
36.	keith	: 4 images
37.	kaiba	: 4 images
38.	mako	: 4 images
39.	yugi	: 4 images
40.	rex	: 4 images
41.	ishizu	: 4 images
42.	joey	: 4 images
43.	jaden—yuki	: 4 images
44.	syrus—truesda	: 4 images
45.	mai	: 3 images
46.	paradox—broth	: 3 images
47.	bastion—misaw	: 3 images
48.	weevil	: 3 images
49.	yami—marik	: 3 images
50.	dr—vellian—c	: 2 images
51.	titan	: 2 images
52.	chazz—princet	: 2 images
53.	axel—brodie	: 2 images
54.	adrian—gecko	: 2 images
55.	yubel	: 2 images
56.	jesse—anderso	: 2 images
57.	alexis—rhodes	: 2 images
58.	zane—truesdal	: 2 images
59.	the—supreme—k	: 2 images
60.	camula	: 2 images
61.	nightshroud	: 2 images
62.	odion	: 2 images
63.	yami—bakura	: 2 images
64.	espa—roba	: 2 images
65.	seto—kaiba	: 2 images
66.	aster—phoenix	: 2 images
67.	tania	: 2 images
68.	amnael	: 2 images
69.	kagemaru	: 2 images
70.	abidos—the—th	: 1 images
71.	unknown	: 1 images
72.	chumley—huffi	: 1 images
73.	thelonious—vi	: 1 images
74.	david	: 1 images
75.	christine	: 1 images
76.	creator—god	: 1 images
77.	joey—wheeler	: 1 images
78.	tyranno—hassl	: 1 images
79.	andrew	: 1 images
80.	arkana	: 1 images
81.	mai—valentine	: 1 images
82.	don—zaloog	: 1 images
83.	tea—gardner	: 1 images
84.	ishizu—ishtar	: 1 images
85.	emma	: 1 images
86.	lumis—and—umb	: 1 images
87.	lumis—umbra	: 1 images

Total: 13281 images
87 categories différentes

Temps : Mise en place du .js : environ 30 min à 1 heure.

2.3.3 Trie de la base :

En vue de la base, j'ai donc de faire le choix de prendre les classes avec 90 images ou plus. En vérité, pour un vrai entraînement, je n'ai que 9 classes "acceptables" qui sont au dessus de 500 images.

J'ai décidé aussi d'enlever la classe counter qui ne concerne que les magies pièges de l'époque de la création du jeu, je les trouvais moins pertinente à entraîner.

Au final, nous retenons 28 classes pour notre entraînement. J'ai donc finalement créer un dernier fichier JS (organize-image.js) qui va me permettre de prendre les classes que je souhaite, et les trier en sous-dossier.

Maintenant que ma base d'image est rangée, cela va être plus simple pour moi de passer sur chaque image afin d'enlever les cartes ne comprenant pas une illustration représentative de sa classe. La limite entre déterminer si une carte correspond ou non à sa classe est fine de plus que le processus est long. J'ai décidé d'en garder quand même la plupart.

Au final, nous travaillerons avec une base d'image total de 12887 images avec 28 classes!

Temps : Mise en place du .js : environ 30 min à 1 heure. Tri manuel de la base 2 à 3 heures.

2.3.4 Mise en place technique - Notebook :

Une fois la base d'images organisée, j'ai centralisé l'entraînement dans un notebook PyTorch unique (`RenduNotebookML.ipynb`) pour garder une logique reproductible de bout en bout : préparation des données, construction des *DataLoaders*, entraînements, évaluations et export des courbes.

Le notebook est structuré par étapes (A à E), avec des paramètres globaux fixés au début (seed, taille d'image, chemins, batch, etc.). Le même prétraitement de base est réutilisé partout (resize en 224x224 + normalisation), puis seules les variables utiles changent d'un essai à l'autre (architecture, optimiseur, scheduler, augmentation).

Pourquoi un split en 3 ensembles (train / validation / test) ?

Le découpage en trois ensembles va permettre d'éviter de se tromper sur la qualité réelle du modèle :

- **Train** : sert à ajuster les poids du modèle (phase d'apprentissage).
- **Validation** : sert à choisir les hyperparamètres (lr, architecture, régularisation, scheduler) et à détecter le surapprentissage sans toucher au test.
- **Test** : sert uniquement à l'évaluation finale, une fois les choix figés, pour obtenir une estimation honnête de la performance en généralisation.

Dans mon cas, ce point est encore plus important car certaines classes peuvent être visuellement proches et le modèle peut exploiter des indices non souhaités (texte, icônes, mise en page des cartes). Le split train/val/test permet donc de vérifier que les gains observés ne proviennent pas uniquement d'un ajustement opportuniste sur la validation.

J'ai utilisé un split **stratifié** 70/15/15 (seed fixe), afin de conserver une proportion de classes similaire dans chaque sous-ensemble. Les tailles réellement obtenues dans les CSV exportés par le notebook sont résumées ci-dessous :

Split	Nombre d'images	Proportion	Nombre de classes
Train	9020	70.0%	28
Validation	1933	15.0%	28
Test	1934	15.0%	28

TABLE 2 – Répartition finale des données (issue des fichiers `train_split.csv`, `val_split.csv`, `test_split.csv`)

Synthèse technique des paramètres notebook :

Paramètre	Valeur utilisée
Dataset nettoyé	12 887 images exploitables, 28 classes
Taille d'entrée	224 x 224
Normalisation	Mean/Std ImageNet
Split	Stratifié 70/15/15 avec seed fixe (42)
Chargement données	<code>DataLoader</code> PyTorch (shuffle train, non-shuffle val/test)
Métriques principales	Loss, Top-1 Accuracy (acc1), Top-3 Accuracy (acc3)
Sorties sauvegardées	Checkpoints modèles + exports automatiques des graphiques

TABLE 3 – Choix techniques appliqués dans le notebook d'entraînement

Cette mise en place m'a permis de comparer les essais de manière cohérente, en changeant un nombre limité de variables à la fois.

3 L'apprentissage :

Pour l'apprentissage, j'ai construit un pipeline avec PyTorch complet en plusieurs étapes :

- une baseline CNN pour avoir un point de référence.
- des essais avec Data Augmentation.
- un tableau d'optimisation d'hyperparamètres pour comparer.
- un test complémentaire avec ResNet18.
- puis un entraînement final afin de voir ce qui a plus ou moins marché.

J'ai conservé la même logique pour tous les essais :

- Evaluer le train/validation pour la loss et l'accuracy.
- images redimensionnées en 224x224.
- loss, top-1 accuracy (acc1) et top-3 accuracy (acc3) pour certains tests.

La majorité des tests ont été réalisés sur GPU (Nvidia GTX 1650 super, bien utilisé...) et d'autre sur CPU sur Mac avec 11 coeurs. Je préciserai à chaque fois pour le temps d'entraînement.

3.1 Essai 0 : Baseline CNN de référence :

Mise en place :

Modèle utilisé : *SmallCNN* (3 blocs convolution + batch norm + ReLU + maxpool, puis classification avec average pooling et dropout).

Paramètres principaux :

- batch size = 16
- epochs = 32
- optimiseur = Adam
- learning rate = 1e-3
- weight decay = 0
- data augmentation = non

Le but ici va être d'obtenir un premier modèle stable, servant de base de comparaison pour tous les essais suivants. Je l'ai entraîné sur plus d'époques afin de voir à partir de quand le surapprentissage apparaît.

Scores / performances obtenus (avant surentrainement epoch=12) :

- val_acc1 : **0.31 (31%)**
- val_acc3 : **0.59 (59%)**
- test_acc1 : **0.30 (30%)**
- test_acc3 : **0.58 (58%)**
- temps d'entraînement : **8111.69 s**, 2h 15min 12s. (GPU)

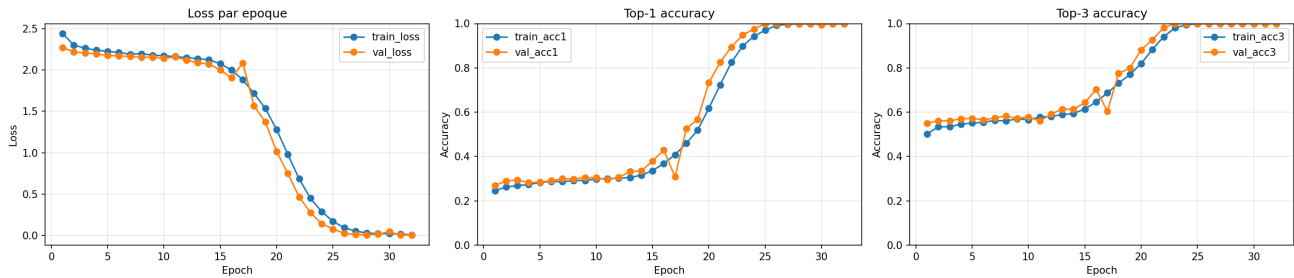


FIGURE 4 – Model 0 : Référence SmallCNN

Problèmes rencontrés :

Le modèle apprend, l'accuracy n'est pas de 1/28, mais il fait quand même pas mal d'erreur. Cela s'explique facilement avec le manque de données pour 60% des classes. Lors de mon test final je regarderai quelle classes sont le mieux apprises.

Conclusion de l'essai :

Baseline utile et cohérente pour la suite, mais peut-on trouver de meilleurs hyperparametres pour optimiser notre modèle.

3.2 Essai 1 : Images de meilleur qualité

Voyons si le fait de rentrer des images de meilleurs qualités change quelque chose. Dans les faits, je resize les images en 242x242, donc on va juste donner à notre modèle de nouvelles données à traiter. Dans la suite des expériences, les données utilisés seront les images ces images-ci.

Scores / performances obtenus (avant surentrainement epoch=10) :

- val_acc1 : **0.29 (31%)**
- val_acc3 : **0.58 (59%)**
- test_acc1 : **0.28 (30%)**
- temps d'entraînement : **8701.89 s**, 2h 20min 2s. (GPU)

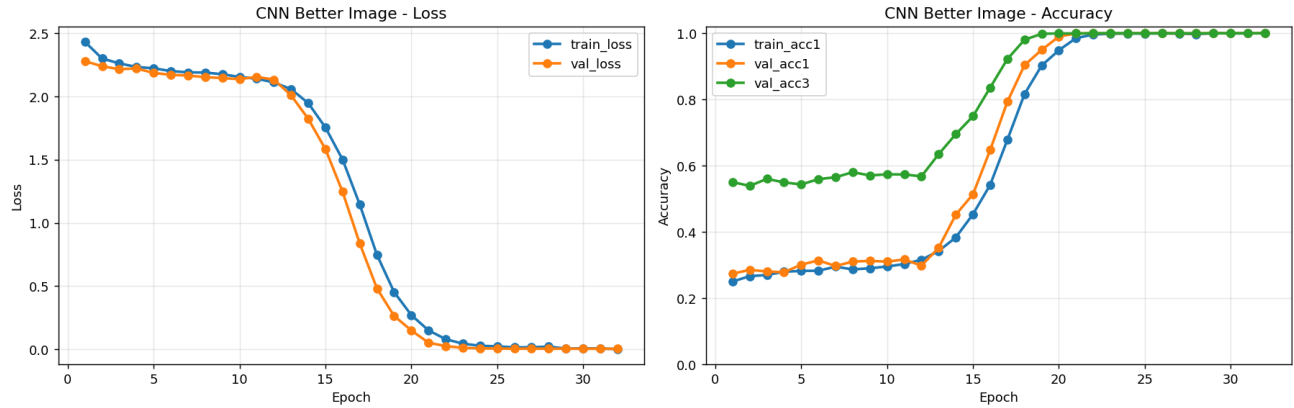


FIGURE 5 – Model 1 : Référence SmallCNN avec better images

En effet, pas de changement majeur mis à part un surapprentissage qui arrive plus rapidement.

3.3 Essai 2 : Test ResNet18 simple

Mise en place :

Un essai complémentaire avec *ResNet18* a été ajouté, en version simple :

- tête finale remplacée pour s'adapter aux 28 classes
- optimiseur Adam, $lr=1e-3$, weight decay= $1e-4$
- 10 epochs

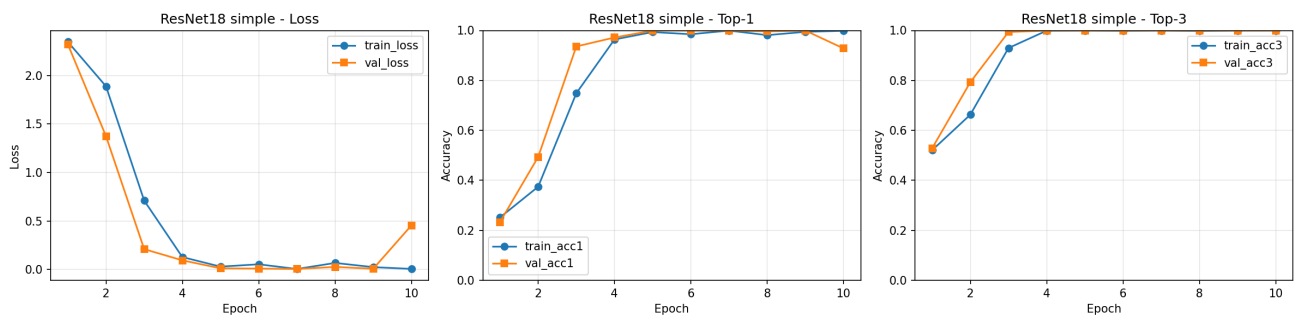
Objectif attendu :

Vérifier rapidement si une architecture résiduelle apporte un gain par rapport aux CNN classique. Notament est-ce que un ResNet va pouvoir détecter le texte de nos cartes.

Même split de données, même normalisation, pas d'augmentation forte pour garder une comparaison.

Scores / performances obtenus pour 10 epochs :

- val_acc1 : **0.95 (99%)**
- test_acc1 : **1.00 (100%)**
- val_acc3 : **1.00 (100%)**
- temps d'entraînement : **3642.1 s**, 1h 00min 42s. (CPU)

FIGURE 6 – Model 2 : ResNET18 Adam $lr=1e-3$

Conclusion de l'essai :

ResNet18 marche parfaitement. Deux solutions, soit j'ai un problème d'entraînement, mais après vérification tout semble être normal, soit le ResNet18 a détecté quelque chose sur mon image qui lui donne la solution. Je partirais sur la deuxième solution. En effet, sur mes cartes, il est noté le nom de la classe auquel appartient l'image. Le ResNet a donc du reconnaître le pattern de texte et s'en sert pour déterminer la classe. Il a du faire pareil pour les cartes MAGIES, leurs classes n'étant différenciables qu'avec un petit symbole. Il fallait s'y attendre qu'un modèle un peu plus compacte trouverait la faille de mes données.

Pour la suite de l'expérience je vais donc essayer d'optimiser mon CNN afin de voir si certaines optimisations peuvent être faites en jouant sur les hyperparamètres.

3.4 Essai 3 : Data Augmentation

Mise en place :

J'ai testé une montée progressive des augmentations :

- none
- rotate_only
- affine_only
- color_only
- combined (affine + jitter + flip)

Objectif attendu :

Améliorer la généralisation du modèle en rendant l'entraînement moins sensible aux variations de position, couleur et orientation.

Contexte complet :

Architecture conservée : SmallCNN. Hyperparamètres proches de la baseline pour isoler l'effet des transformations.

Scores / performances obtenus :

Mode d'augmentation	val_acc1	test_acc1	Remarque
none	0.310	0.299	référence
rotate_only	0.312	0.301	léger gain, mais instable selon les classes
affine_only	0.314	0.304	gain faible mais plus régulier
color_only	0.309	0.298	quasi identique à la référence
combined	0.316	0.306	meilleur compromis, amélioration modeste

Comparaison des performances avec/sans :

Il n'y a pas de réels changements. Mes images sont très variées et disparates, la data augmentation ne permettrait pas de couvrir plus de cas que l'on retrouverait dans la partie val ou test de mon dataset.

3.5 Essai 4 : Optimisation des hyperparamètres (HPO)

Mise en place :

Tous les entraînements suivants ont été faits sur GPU. J'ai mis en place une grille d'expériences a été lancée sur 4 architectures de CNN différentes : SimpleNet, SmallCNN, MediumCNN, LargeCNN. On fera trois changements d'hyperparamètre par modèle, à part pour le plus large qui est plus long à entraîner.

Hyperparamètres testés :

- learning rates : 1e-3, 5e-4, 1e-2, 5e-3
- optimiseurs : Adam, AdamW, SGD
- batch size : 16 ou 32
- dropout : 0.25 à 0.40
- weight decay : 0 ou 1e-4
- scheduler : aucun, cosine, reduce_on_plateau

Objectif attendu :

Identifier la meilleure combinaison modèle + optimisation pour voir de un s'il y a un gain et de deux trouver celui qui nous donnera un meilleur modèle.

Scores / performances obtenus :

idx	experiment	model_type	optimizer	lr	batch_size	dropout	weight_decay	scheduler	epochs	val_acc1	val_acc3	test_acc1	test_acc3	train_time_s
0	hpo_medium_adam_lr1e-3	medium	adam	0.0010	32	0.35	0.0000	-	12	0.999488	0.999488	0.999488	1.000000	859.632106
1	hpo_large_adamw_lr5e-4	large	adamw	0.0005	16	0.40	0.0001	reduce_on_plateau	12	0.999483	0.999483	0.999483	0.999483	1213.155945
2	hpo_large_adam_lr1e-3	large	adam	0.0010	16	0.40	0.0001	-	12	0.999483	1.000000	0.999483	1.000000	1237.407921
3	hpo_medium_adamw_lr5e-4	medium	adamw	0.0005	32	0.35	0.0001	reduce_on_plateau	12	0.986168	0.999488	0.983607	0.999488	847.709857
4	hpo_medium_sgd_lr5e-3	medium	sgd	0.0050	32	0.35	0.0001	cosine	12	0.321091	0.598715	0.307816	0.585553	855.154090
5	hpo_small_adam_lr1e-3	small	adam	0.0010	32	0.30	0.0000	-	12	0.309820	0.573849	0.294131	0.582260	872.024088
6	hpo_small_sgd_lr1e-2_cosine	small	sgd	0.0100	16	0.30	0.0001	cosine	12	0.309083	0.579466	0.290511	0.569731	797.078158
7	hpo_small_adamw_lr5e-4_wd	small	adamw	0.0005	32	0.30	0.0001	reduce_on_plateau	12	0.306510	0.579524	0.288349	0.570184	798.345123
8	hpo_simple_adam_lr1e-3	simple	adam	0.0010	32	0.25	0.0000	-	12	0.293939	0.564392	0.275542	0.551595	776.367091
9	hpo_simple_sgd_lr1e-2	simple	sgd	0.0100	32	0.25	0.0001	cosine	12	0.292402	0.557732	0.270419	0.541862	816.257051
10	hpo_simple_adam_lr5e-4	simple	adam	0.0005	32	0.25	0.0000	-	12	0.283457	0.562342	0.267345	0.546107	833.337967

TABLE 5 – Resultats HPO

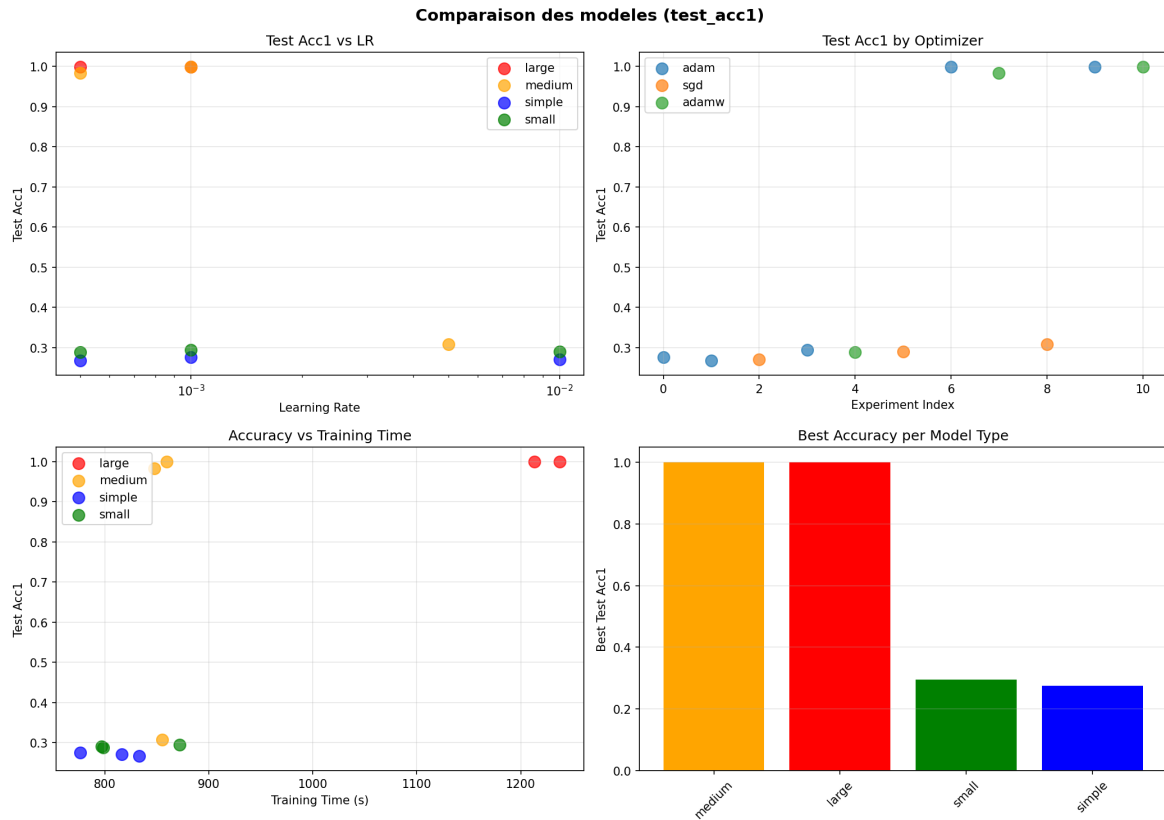


FIGURE 7 – Analyse des différentes HPO

(VOIR ANNEXE PAGE 14 ET 15)

Analyse et conclusions :

Les résultats HPO montrent une rupture nette entre deux groupes de modèles.

Le premier groupe (SimpleNet et SmallCNN) reste dans une zone de performance cohérente avec la baseline, autour de 27% à 31% en *test_acc1*. Cela confirme que ces architectures arrivent à apprendre des motifs utiles, mais restent limitées pour capter toute la variabilité visuelle de la base.

Le second groupe (MediumCNN et LargeCNN avec Adam/AdamW) atteint des scores quasi parfaits (jusqu'à 99.95% en *test_acc1* et 100% en *test_acc3*). Le saut est trop important pour être expliqué uniquement par un gain de capacité classique. Ce comportement est cohérent avec ce qui a déjà été observé avec ResNet18 : les modèles les plus capables exploitent très probablement des indices très faciles présents sur les cartes (texte de type, symboles, structure graphique).

Analyse spécifique des modèles larges :

Les deux LargeCNN testés sont très performants, mais leur coût est plus élevé :

- **hpo_large_adam_lr1e-3** : *test_acc1*=0.999483, *test_acc3*=1.000000, temps ~1237 s.

- **hpo_large_adamw_lr5e-4** : *test_acc1*=0.999483, *test_acc3*=0.999483, temps ~1213 s.

Ils n'apportent pas de gain significatif par rapport au meilleur MediumCNN, tout en demandant davantage de temps d'entraînement. En pratique, ce sont donc de très bons candidats en score brut, mais moins intéressants en compromis performance/coût.

Surentraînement :

Sur les configurations faibles (Simple/Small), on observe surtout un sous-ajustement relatif (plateau bas), plus qu'un surentraînement massif.

Sur les meilleures configurations (Medium/Large), l'écart validation/test est très faible, donc il n'y a pas de surentraînement visible au sens classique (train très haut, val/test qui chutent). En revanche, il existe un risque de **surapprentissage des indices faciles du dataset** (texte et symboles), ce qui peut donner une généralisation artificiellement optimiste sur ce protocole de split.

Choix du meilleur modèle :

Le meilleur compromis retenu est :

hpo_medium_adam_lr1e-3

Si on voulait prendre le meilleur CNN, on aurait :

- Un meilleur score global mesuré (*val_acc1*=0.999488, *test_acc1*=0.999488, *test_acc3*=1.000000),
- temps d'entraînement inférieur aux modèles large (~860 s),
- architecture moins coûteuse à entraîner et plus simple à réutiliser.

3.6 Essai final : Analyser comment un small CNN reconnaît notre base :

Dans les faits, nous avons trouvé des modèles presque parfaits (Medium/Large), mais avec un risque fort d'exploitation d'indices faciles (texte, icônes et mise en page). Pour comprendre ce que le modèle apprend réellement sur l'illustration, j'ai choisi d'analyser un **SmallCNN** en détail avec les sorties de la section E (analyse classe par classe).

Objectif de cet essai final :

- mesurer quelles classes sont réellement apprises par un modèle de capacité plus limitée,
- identifier les classes systématiquement confondues,
- vérifier si les bonnes performances globales observées sur d'autres modèles sont cohérentes avec le contenu visuel du dataset.

Résultats globaux :

On observe donc un modèle très **déséquilibré** : il reconnaît extrêmement bien une classe dominante (*Normal*) mais reste faible, voire nul, sur de nombreuses classes minoritaires ou visuellement proches.

Classes les mieux reconnues vs les plus difficiles :

Indicateur	Valeur
Nombre total d'images testées	1934
Nombre de classes évaluées	28
Accuracy top-1 pondérée (SmallCNN)	0.292
Classes avec accuracy nulle	16 / 28
Meilleure classe	Normal (0.997)

TABLE 6 – Synthèse de l'analyse finale SmallCNN

Top classes (accuracy élevée)			Classes en échec (accuracy faible)		
Classe	Support	Acc.	Classe	Support	Acc.
Normal	336	0.997	Wyrn	14	0.000
Warrior	163	0.436	Thunder	21	0.000
Cyberse	41	0.415	Dinosaur	20	0.000
Fiend	128	0.359	Sea-Serpent	13	0.000
Fairy	82	0.317	Reptile	27	0.000

TABLE 7 – Contraste de performance par classe pour le SmallCNN

Interprétation par rapport au dataset :

Cette répartition confirme le problème principal de la base : certaines classes bénéficient d'indices visuels forts et répétitifs (couleur dominante, structure de carte, mise en forme du texte), alors que d'autres classes sont plus fines et partagent des caractéristiques visuelles proches. Le SmallCNN a tendance à apprendre d'abord les signaux les plus simples, et peine à séparer les classes plus ambiguës.

En pratique, cela signifie que :

- la performance moyenne cache une forte variabilité inter-classes,
- les classes peu représentées ou visuellement proches restent mal reconnues,
- une accuracy globale seule n'est pas suffisante pour juger la robustesse réelle du modèle.

Graphiques Data :

(VOIR ANNEXE PAGE 16)

Conclusion de l'essai final :

L'analyse du SmallCNN montre qu'il ne "comprend" pas uniformément les 28 classes : il réussit très bien certaines classes dominantes, mais échoue sur une grande partie des classes fines. Cela renforce l'idée que les scores quasi parfaits des modèles plus grands proviennent en partie de la structure du dataset et des indices faciles. Pour une évaluation plus robuste, il faudra limiter ces indices (masquage texte/icônes, split plus strict, et équilibrage des classes).

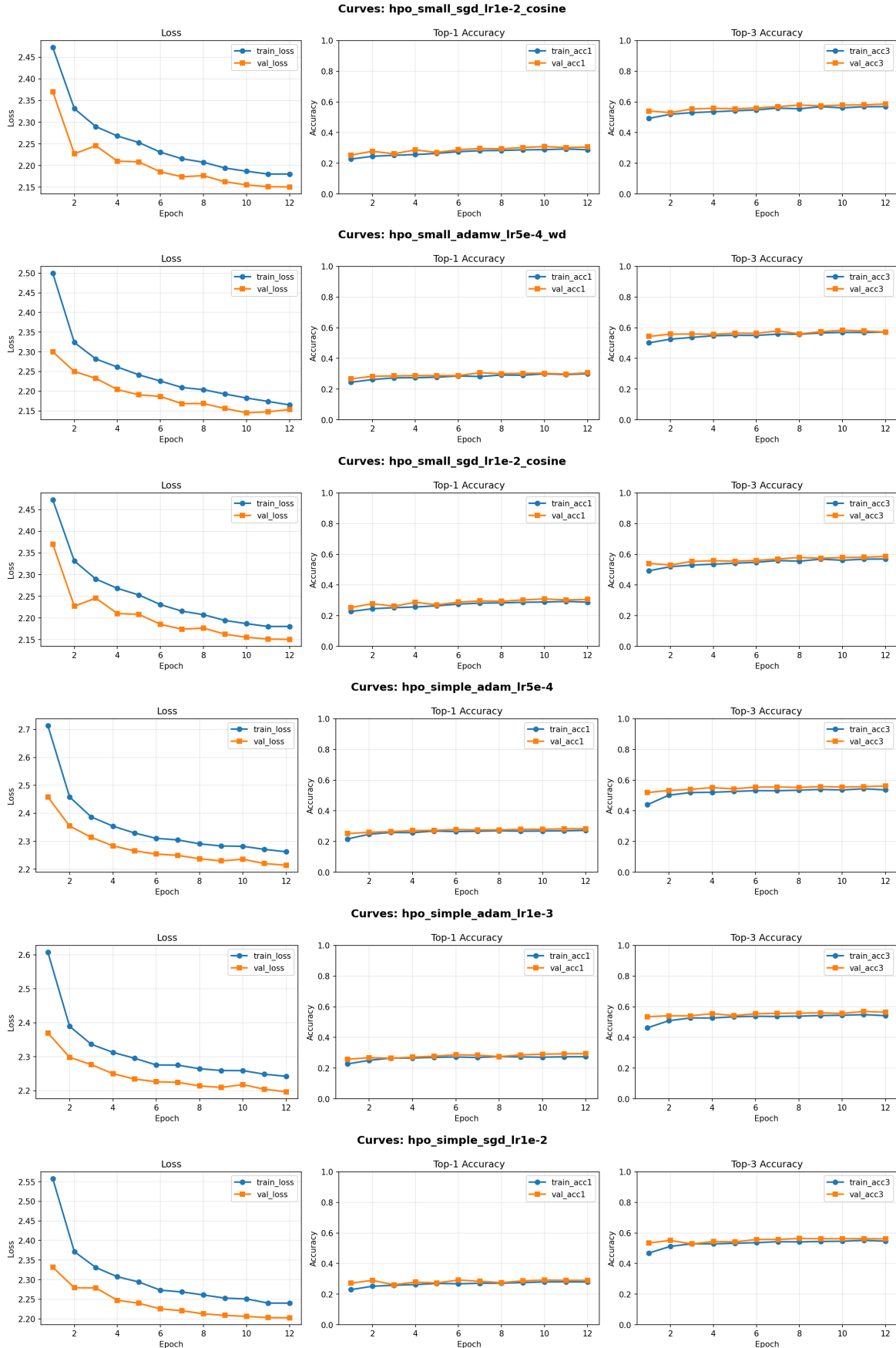


FIGURE 8 – HPO Small and Simple CNN

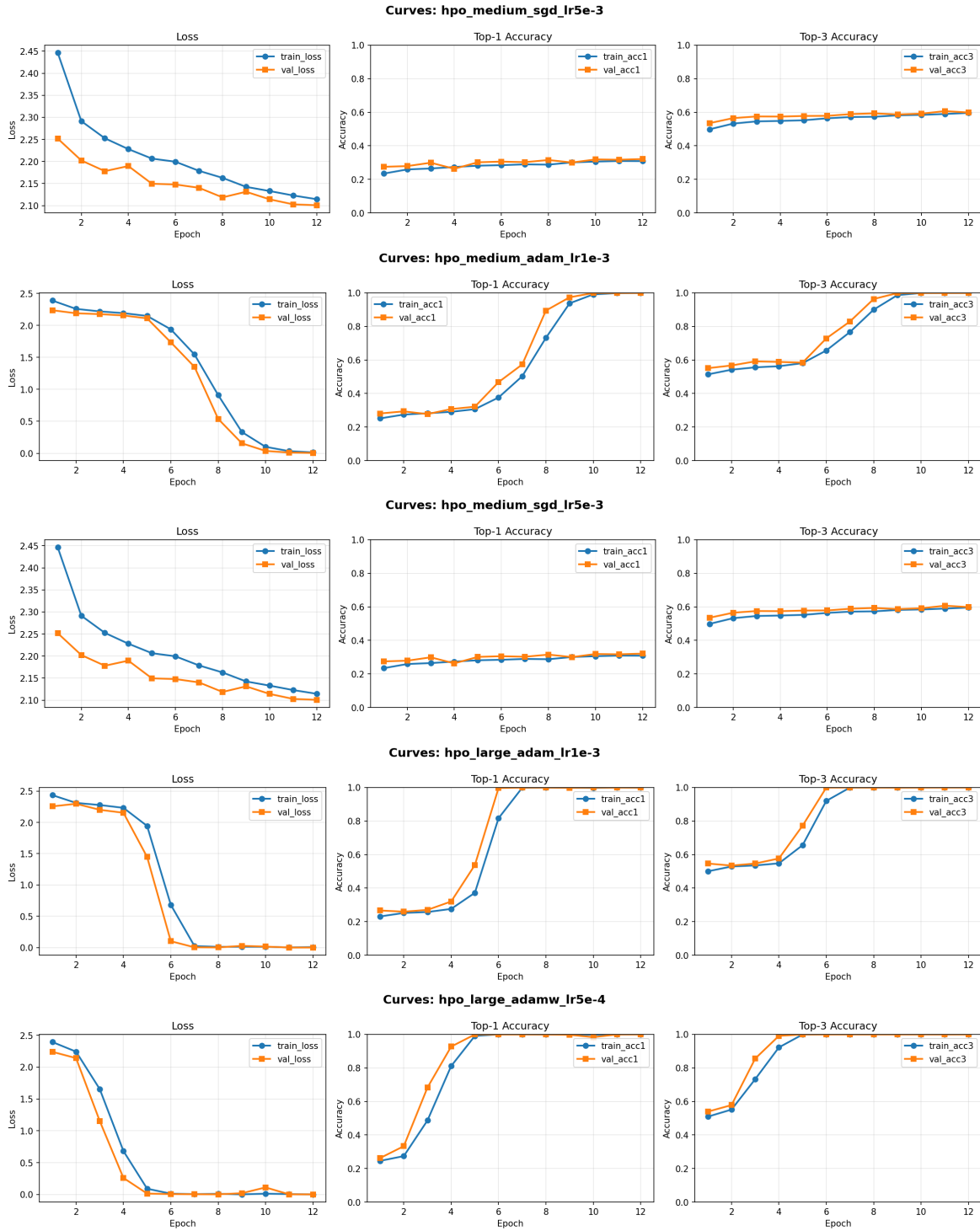


FIGURE 9 – HPO Medium and Large CNN

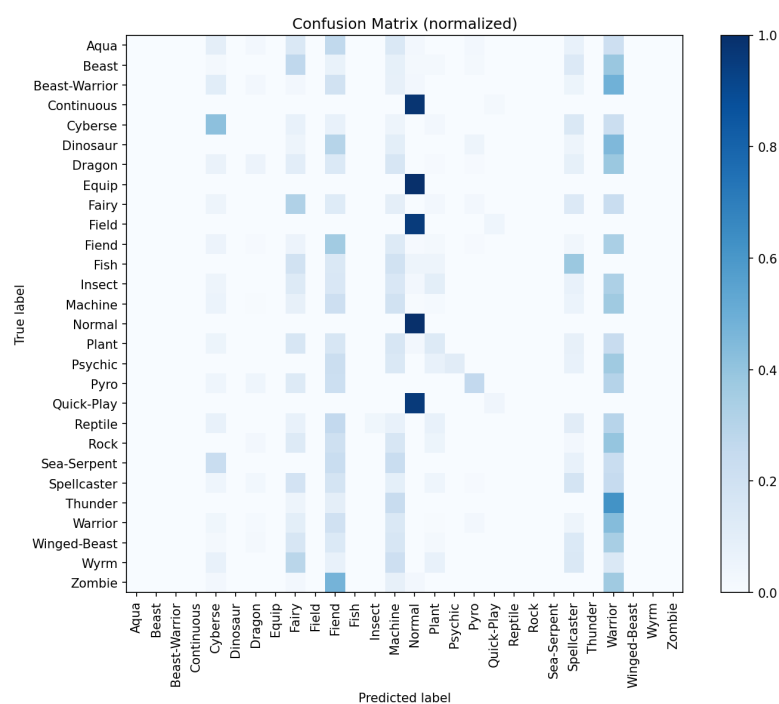


FIGURE 10 – Analyse des différentes classes - 1

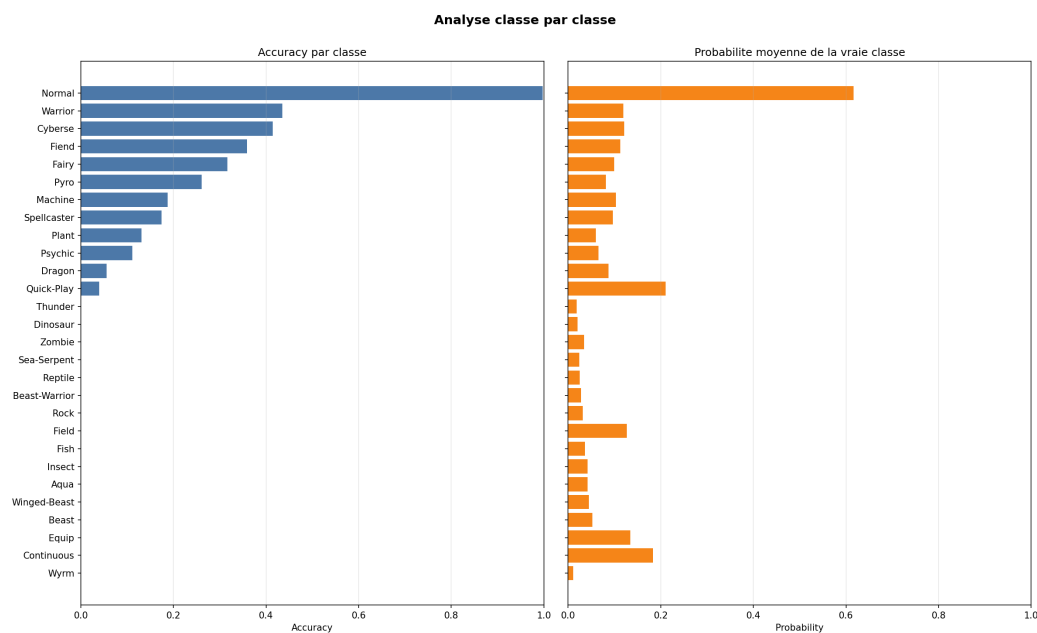


FIGURE 11 – Analyse des différentes classes - 2